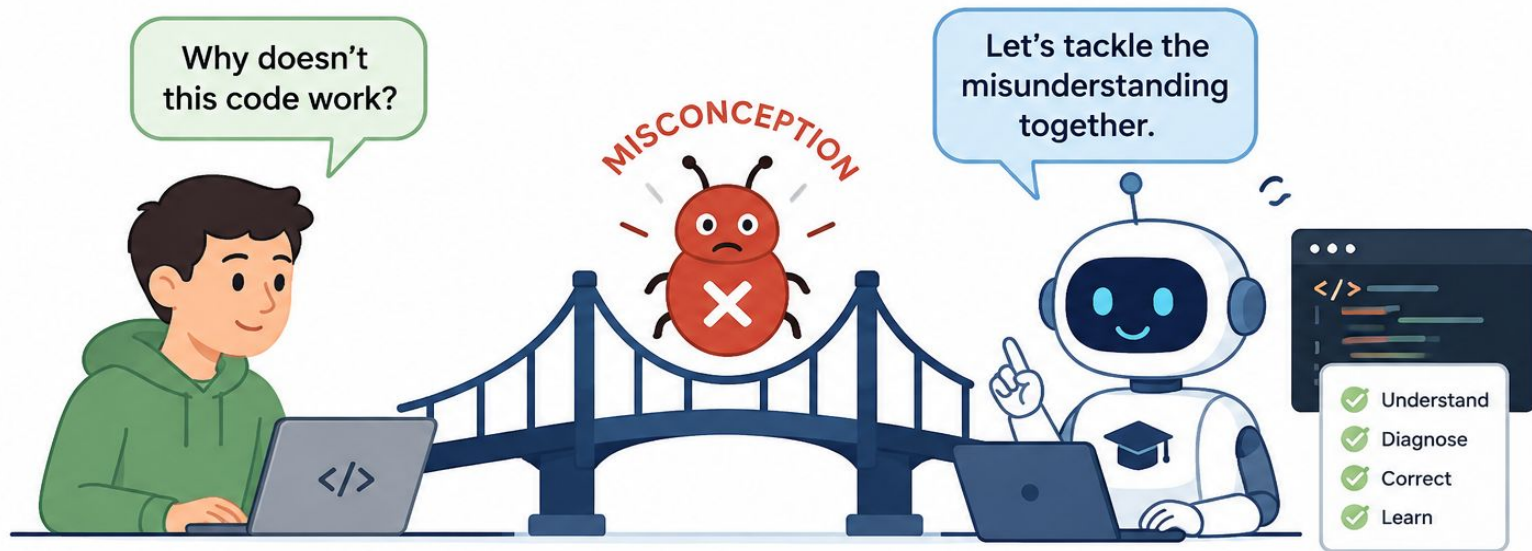


Bridging Misconception Detection and Dialogue-Based Tutoring



Catherine Awad • Ruiwen Guan • Hanieh Hedayati



Motivation

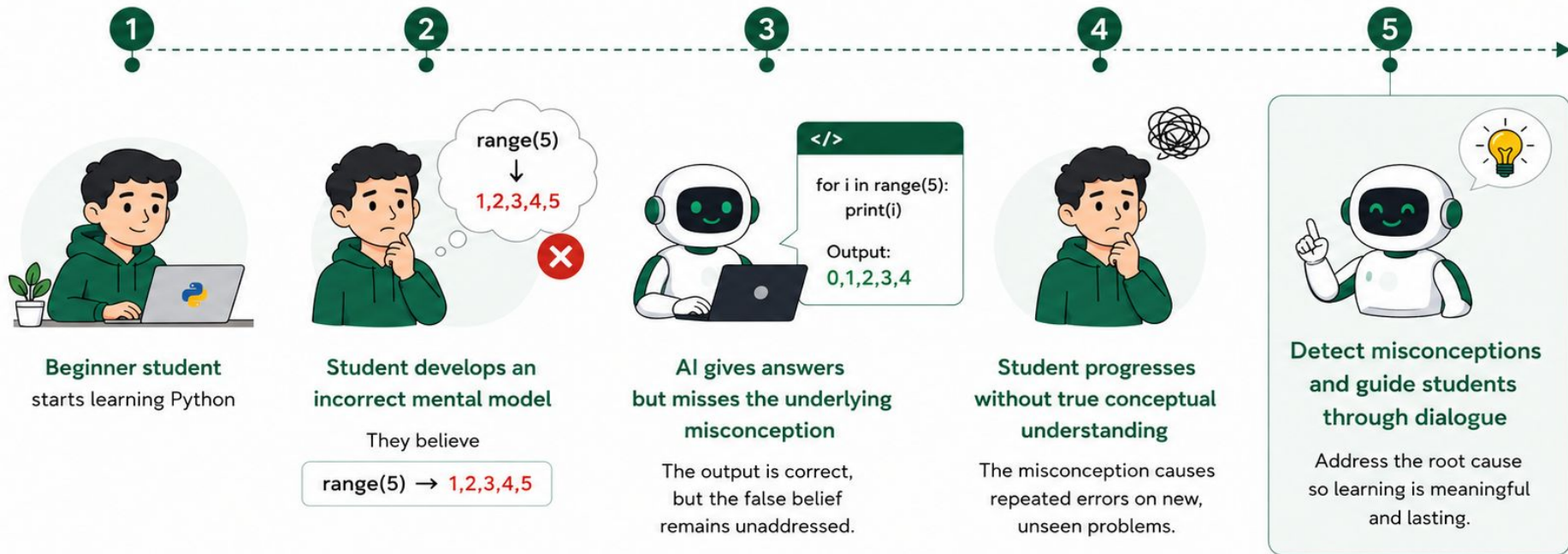


Why this problem matters



The Story: Why Current AI Tutors Fall Short

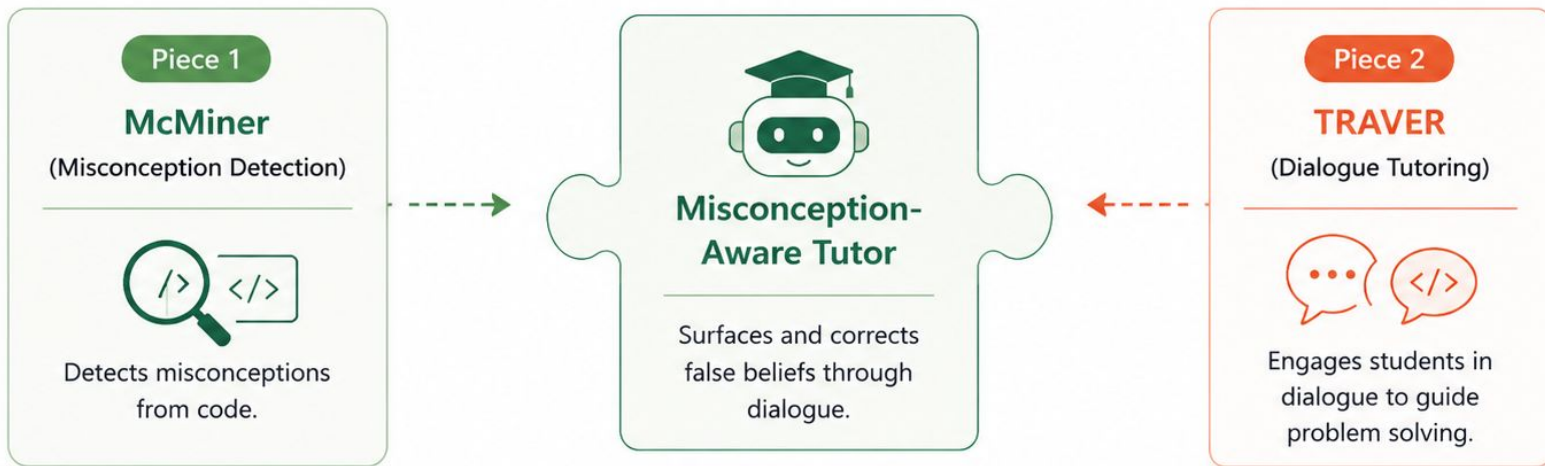
A student can appear to make progress while the underlying misconception remains.



Without addressing misconceptions, students don't truly learn—they just move forward with faulty understanding.

Motivation & Research Objective

Our goal is to build a tutor that **surfaces and corrects misconceptions** before progressing with the task.



Objective: Combine McMiner's diagnostic power with TRAVER's dialogue framework to create a **misconception-aware tutor**.



Methodology



How we build our approach



Existing Systems: Two Complementary Pieces

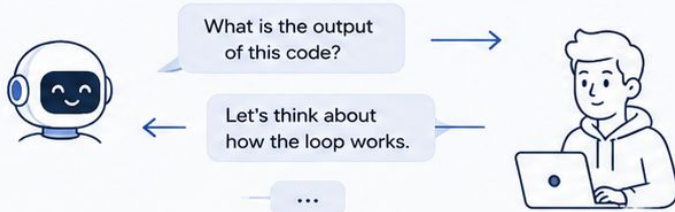
We build on two complementary lines of work—yet current systems keep them separate.



TRAVEL

(Dialogue-Based Tutoring)

Engages students in Socratic dialogue to guide problem solving.



Strength: Great at guiding through steps and encouraging reasoning.



McMiner

(Misconception Detection)

Analyzes code to detect specific misconceptions automatically.



Strength: Precise and scalable detection of misconception patterns.



The Gap: These systems solve different parts of the problem—but don't work together.



Our goal: Integrate misconception detection into dialogue tutoring to surface and correct false beliefs during the learning process.

Original TRAVER Pipeline

A dialogue-based tutoring loop followed by posttest code generation and evaluation.



1 Dialogue Phase (TRAVER)

Tutor and student engage in a dialogue to solve the problem.



Tutor



Student



2 Posttest Code Generation & Evaluation

We generate a posttest and evaluate student performance.



Posttest
Prompt



Student
Code



Evaluation
(Pass / Fail)



Outcome: Pass / Fail on the posttest determines whether the student has successfully learned the concept.



What TRAVER Does Well:

Guides students through dialogue and provides feedback until they are ready for the final challenge.



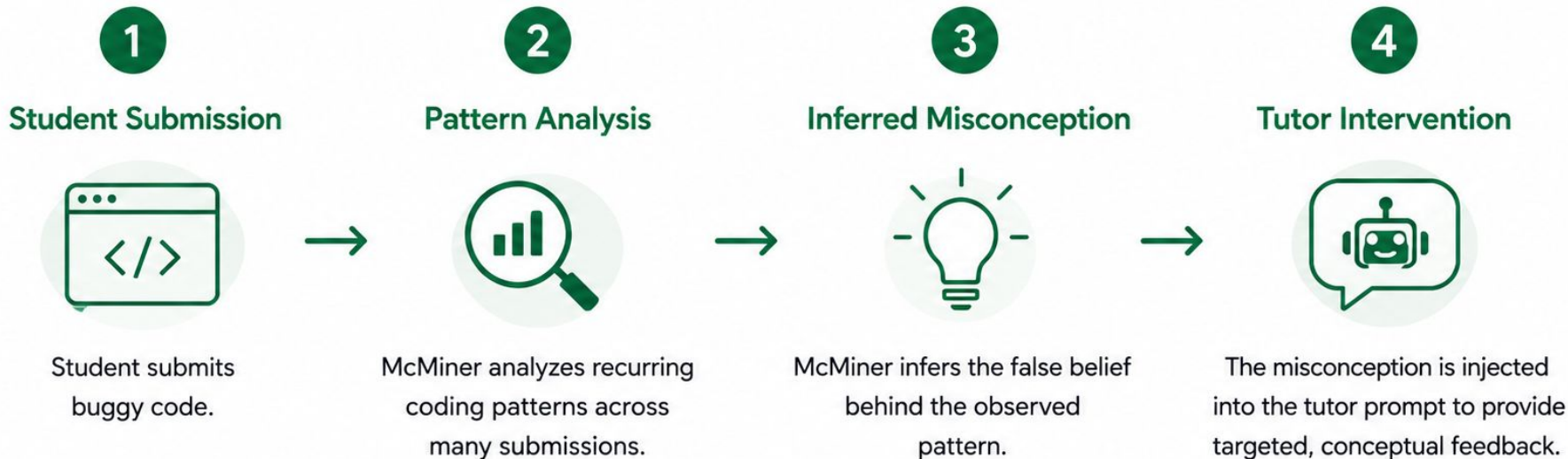
What It Does Not Do:

TRAVER does not detect misconceptions explicitly during the dialogue.

How McMiner Detects Misconceptions



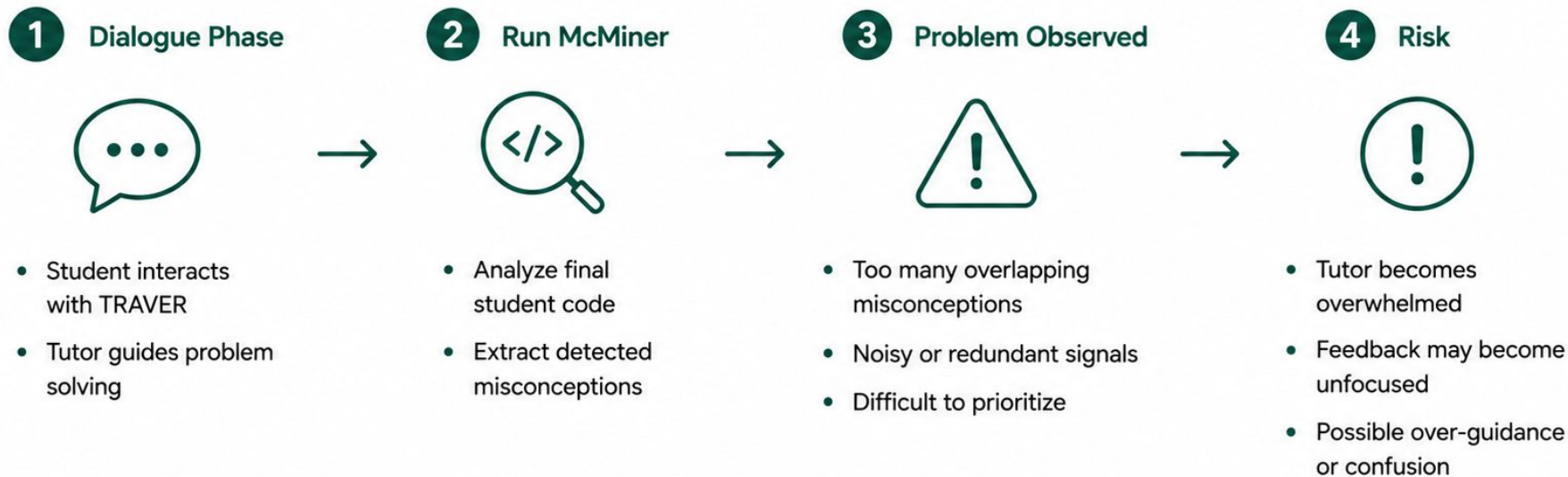
From buggy code → inferred false belief



First Attempt: Batch Misconception Injection



Use TRAVER normally → run McMiner once → inject all detected misconceptions back to TRAVER.



Key Insight

Instead of injecting everything at once:

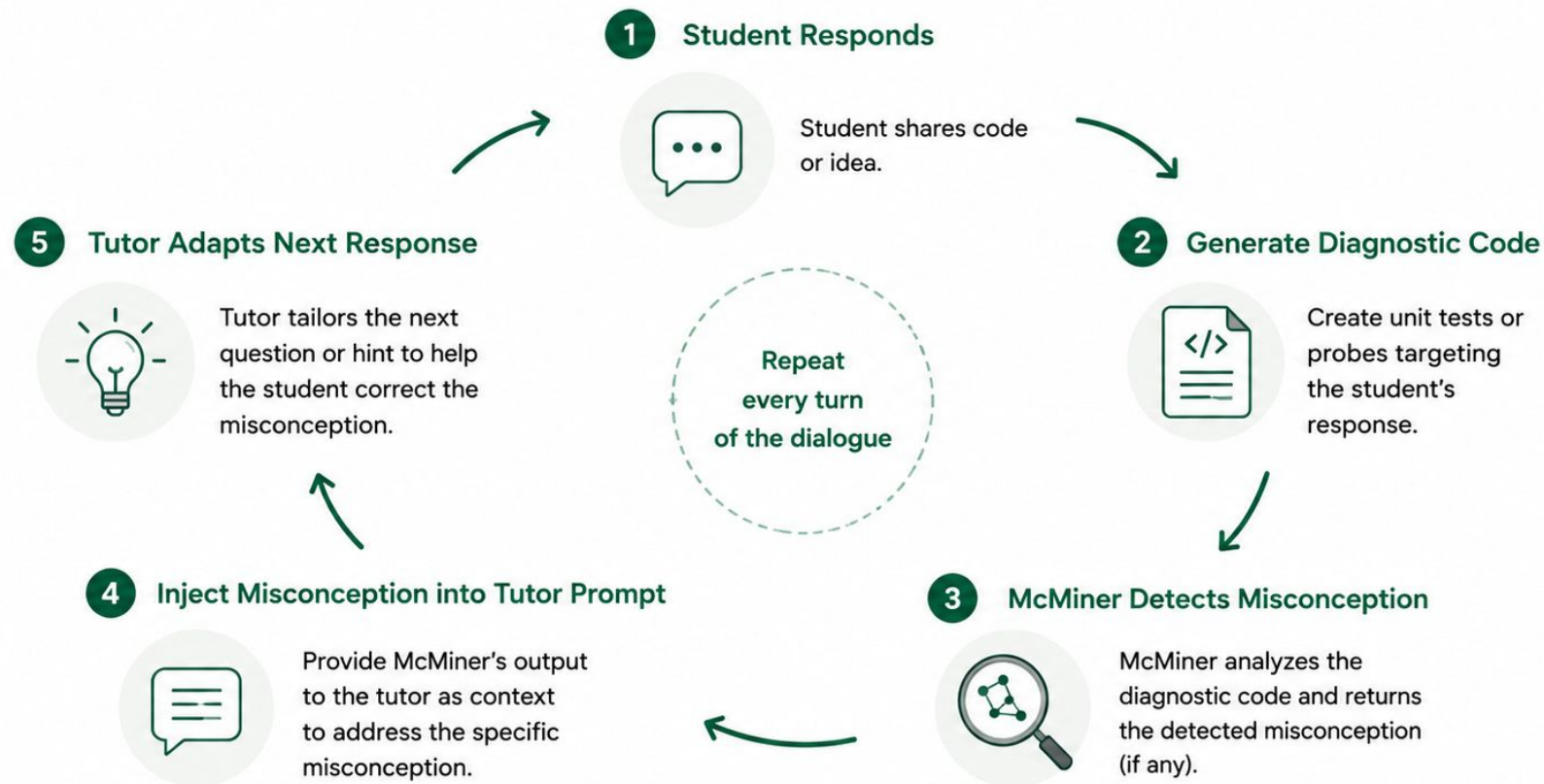
→ **Integrate McMiner directly inside the tutoring loop**, one misconception at a time.



Targeted
Timely
Effective

McMiner-in-the-Loop

Integrate McMiner into the dialogue loop to detect and address misconceptions in real time.

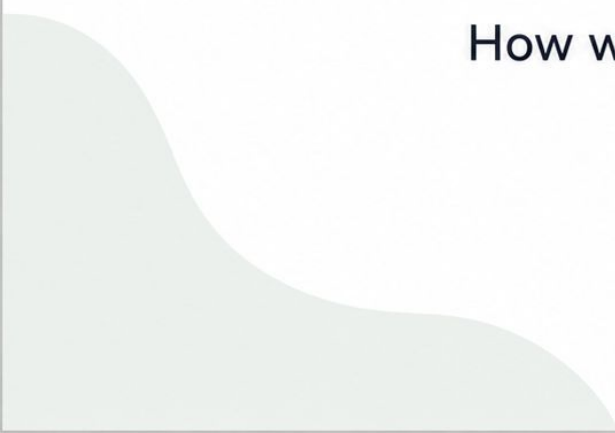




Evaluation & Results



How we evaluated and what we found



Experimental Setup: Ours vs. Original TRAVER



We reproduce the TRAVER baseline under limited computational resources.

Component		TRAVER (Original)	Ours (Reproduction)
	Repositories	25	4
	Tasks	100	22
	Quantization	FP16	4-bit NF4
	Model / Setting		
	Student (dialogue)	Mixtral-8x7B	Mistral-7B
	Student (codegen)	Mixtral-8x7B	Llama-3.1-8B
	Tutor	Llama-3.1-70B	Llama-3.1-70B
	Verifier	Mistral-7B+LoRA	Mistral-7B+LoRA



Key Takeaway: We reproduce the TRAVER baseline using fewer repositories and tasks, and more aggressive quantization, all under limited computational resources.

Baseline Results: Best-round Pass@k (%) per Project and Level

We report best-round Pass@k (%) for TRAVER (original) vs. our reproduction.



Project	Level	TRAVER (Original)		Ours (Reproduction)	
		P@1	P@10	P@1	P@10
sd-forge	low	43.3	100	86.7	100
	med	83.3	100	90.0	100
	high	40.0	100	86.7	100
UHGEval	low	20.0	100	100	100
	med	40.0	100	90.0	100
	high	30.0	100	90.0	100
searcharray	low	3.3	16.7	0.0	0.0
	med	5.0	16.7	0.0	0.0
	high	0.0	0.0	15.0	25.0
EasyVolcap	low	23.3	33.3	23.0	23.3
	med	14.2	25.0	16.0	30.0
	high	23.3	33.3	15.8	33.3
Average (across all projects)		20.8	37.9	27.5	38.0



Key Takeaway: Our reproduction matches or outperforms the original TRAVER baseline across most projects and levels, despite using fewer resources and a reduced benchmark.

Over-Tutoring Problem (Baseline Only)

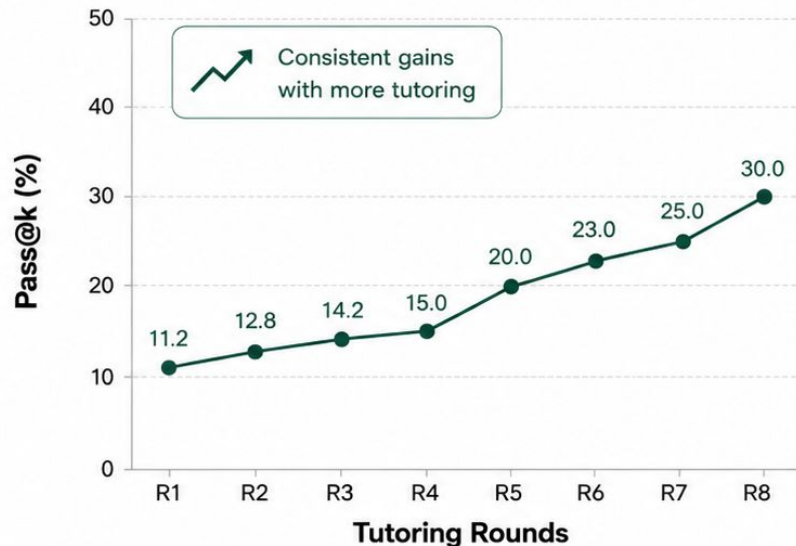
Baseline tutoring helps low-level students, but hurts high-level students.

—●— Baseline (BL)



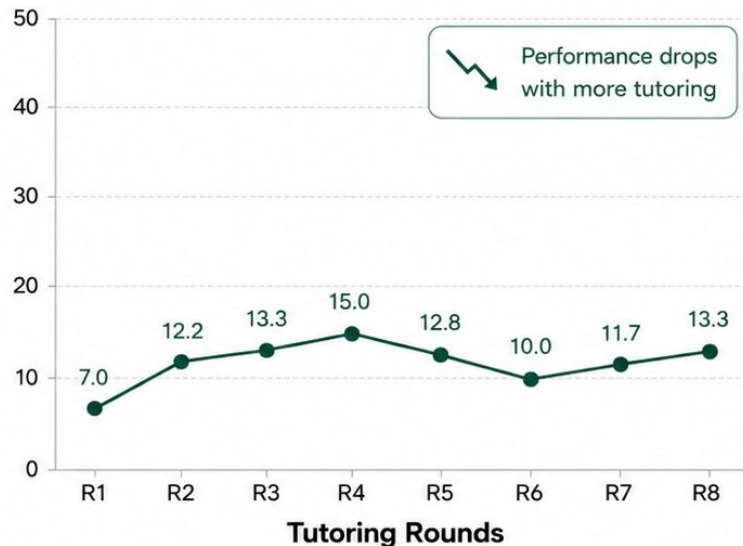
Low-Level Students

Performance improves as rounds increase



High-Level Students

Performance degrades as rounds increase

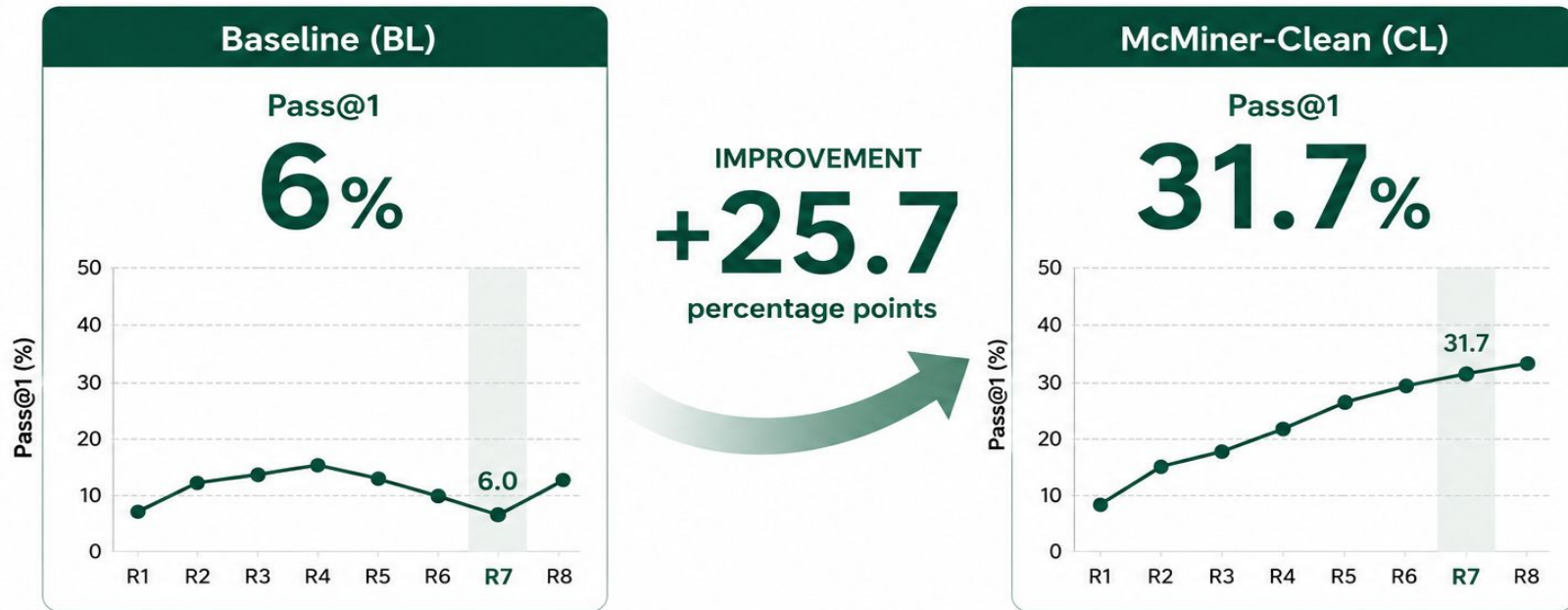


Key Takeaway: More tutoring is not always better. Baseline tutoring hurts high-level students.

Headline Result

McMiner-Clean solves the over-tutoring problem for high-level students.

EasyVolcap – High Level (R7)



McMiner-Clean dramatically improves high-level student performance.

From **6%** → **31.7%** at R7

Results Across All Projects

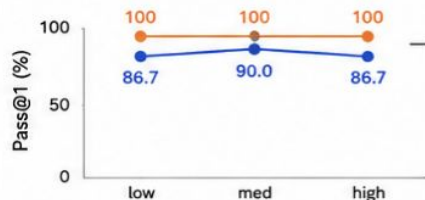
—●— Baseline (BL) —●— McMiner (MC) - -●- - McMiner-Clean (CL)

1 sd-forge



Nearly Saturated

All methods perform very well.



Best Pass@1 (%)	
Baseline (BL)	86.7
McMiner (MC)	100.0
McMiner-Clean (CL)	—

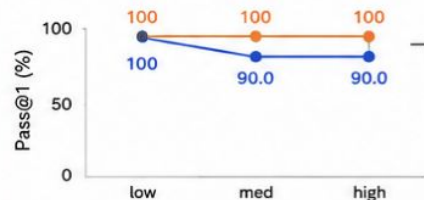
✓ All methods near ceiling.

2 UHGEval



Strong Across the Board

High performance for all methods.



Best Pass@1 (%)	
Baseline (BL)	90.0
McMiner (MC)	100.0
McMiner-Clean (CL)	—

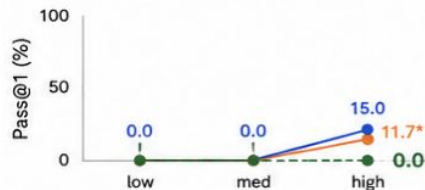
✓ McMiner achieves perfect scores.

3 searcharray



Still Challenging

Low overall performance remains.



Best Pass@1 (%)	
Baseline (BL)	15.0
McMiner (MC)	11.7*
McMiner-Clean (CL)	0.0

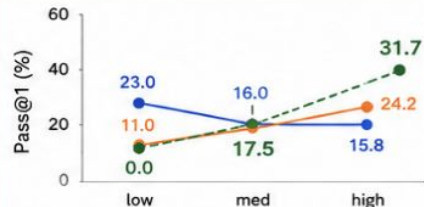
! Problem remains difficult for all methods.

4 EasyVolcap



Largest Improvement

McMiner-Clean delivers major gains.

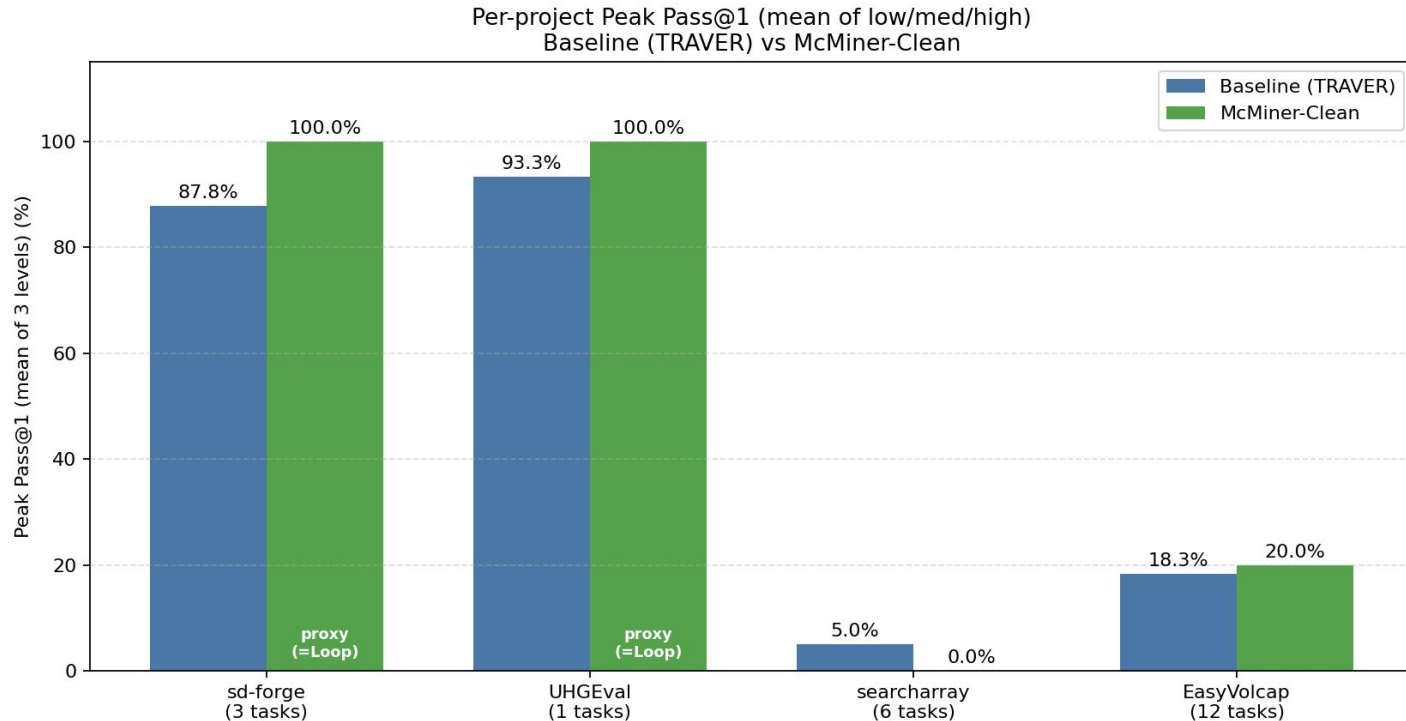


Best Pass@1 (%)	
Baseline (BL)	15.8 (R1)
McMiner (MC)	24.2 (R7)
McMiner-Clean (CL)	31.7 (R7)

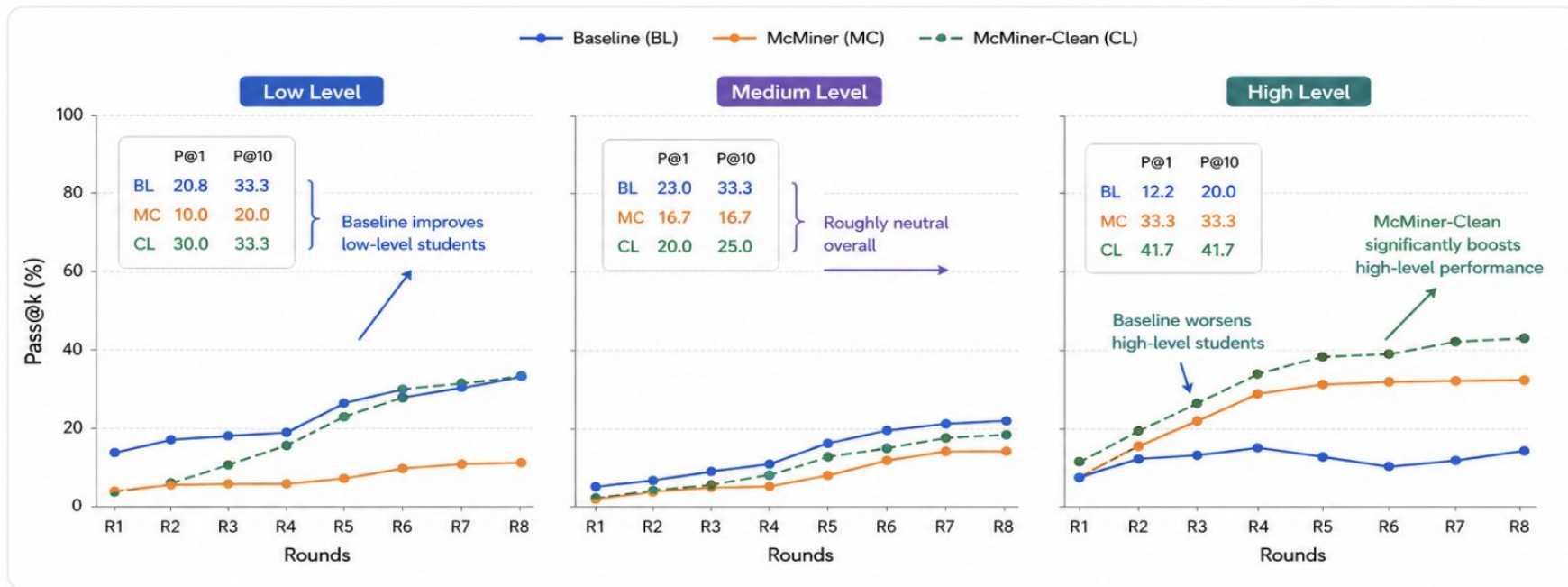
↑ McMiner-Clean achieves +15.9 pp over baseline at high level!

* McMiner-Thorough variant (12 rounds + dedup).

Overall Comparison for Pass@1



EasyVolcap: Tutoring Hurts High-level, McMiner-Clean Helps



Baseline tutoring helps low-level students.

Baseline tutoring hurts high-level students.

McMiner-Clean dramatically improves high-level performance.

Ablation Study: Scaffolded TRAVER

The Idea

✗ Not:

“You have misconception X.”

✓ Instead:

- What are the inputs?
- What should the output be?
- Which API matters?
- Fill one small part at a time.

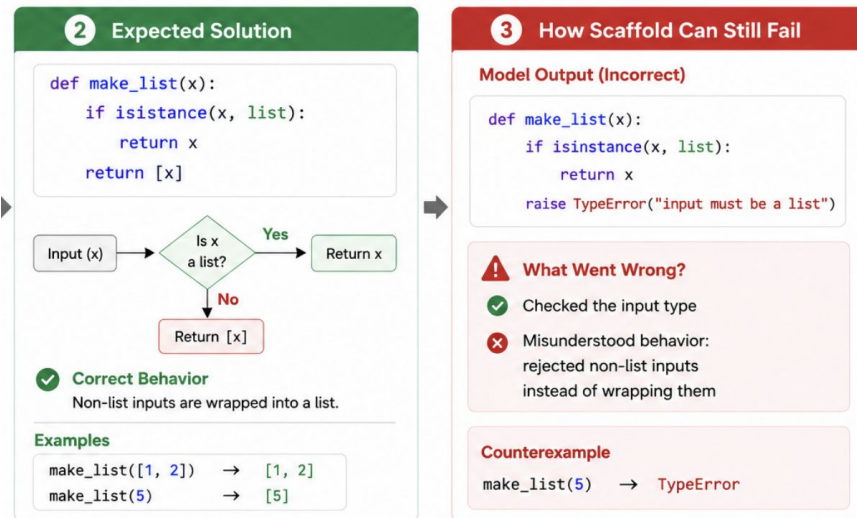
What it is NOT

- Not a new model
- Not a new benchmark

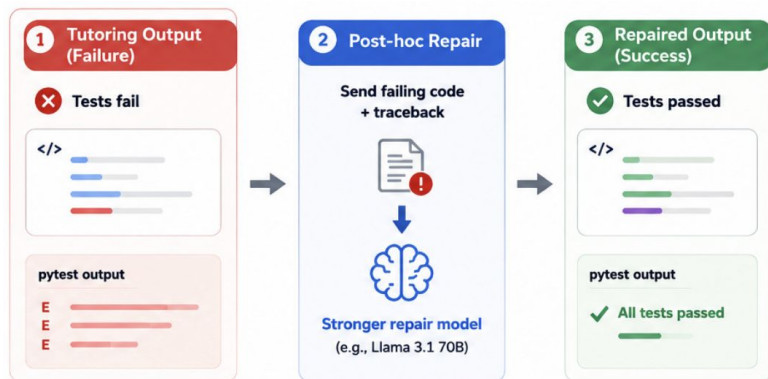
➡ Just a prompt/pipeline modification on top of TRAVER.

What I Changed

- Added a scaffold policy to the tutor prompt
- Added compact API hints from metadata
- Allowed more tutoring rounds
- Kept the same tasks and evaluation setup

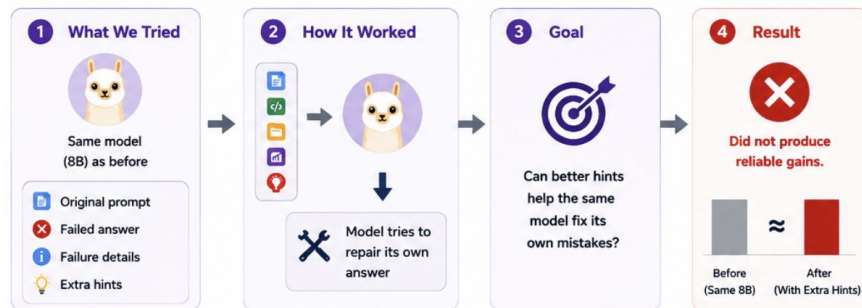


Post-hoc Repairing of Model Outputs



Results:

Project	Level	TRAVES P@1	Repair P@1	Improvement
EasyVolcap	low	23.0%	25.3%	+2.3%
EasyVolcap	med	16.0%	16.8%	+0.8%
searcharray	med	0.0%	22.2%	+22.2%



Takeaway: Extra context and hints alone were not enough — stronger reasoning capability matters.



4

Limitations & Conclusion

What we learned, what's next, and the key takeaways



Limitations & Future Work

Current Limitations



Compute Constraints

- Reduced benchmark size
- Smaller student model and limited compute budget



Task Scope

- Evaluated on coding tutors only
- Limited repositories and problem domains



Student Adaptivity

- Fixed tutoring strategy
- Not dynamically personalized yet

Future Directions



Hybrid Systems

- Combine misconception-aware tutoring with execution-based repair
- End-to-end adaptive learning systems



Adaptive Tutoring Policies

- Detect when and how much to intervene
- Learn student-specific policies to avoid over-tutoring



Larger Evaluation

- More repositories, tasks, and difficulty levels
- Real student studies to assess learning outcomes

❖ **Overall, our results suggest that effective AI tutoring requires not only structured guidance, but also strong execution-aware reasoning and adaptive feedback.**

Conclusion



1

Misconceptions Matter

Targeted, misconception-aware tutoring improves reasoning quality and student performance.



2

More Tutoring Is Not Always Better

Excessive intervention can hurt advanced learners through over-tutoring.



3

Adaptive Tutoring Drives Better Outcomes

Adaptive strategies consistently improve high-level student performance across tasks.

Main Takeaway: Future tutoring systems should adapt intervention depth to each student's understanding.

Q&A

Discussion & Questions

1. Why might an approach work well on one project like EasyVolcap but not transfer as well to another codebase?
2. We injected mcminer in the loop because a lot of misconceptions would have confused and overwhelmed the model, steered it away from the main context. Would breaking down the misconceptions at first and feeding them task by task at the start of the dialog worked better than getting the misconception round by round?
3. The code repair worked when we used a bigger 70B llama model, but it didn't work with the smaller model. Does this mean that the bigger llama fixed mistakes in the student's reasoning or just generated better version of the code?
4. Do you think adding extra rounds will help the students to understand better?

THANK YOU!